

Algoritmos em Grafos

Busca em Largura

Exercícios

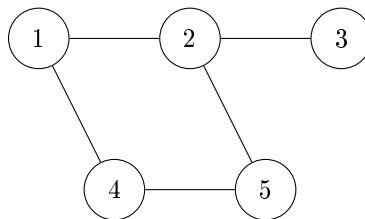
Teoria e Implementação em Java

Material Didático

Exercício Teórico

Exercício 1 — Distância e predecessor na BFS

Considere o grafo não orientado G a seguir, com 5 vértices:



As listas de adjacências, em ordem crescente, são:

$$Adj[1] = (2, 4), \quad Adj[2] = (1, 3, 5), \quad Adj[3] = (2),$$

$$Adj[4] = (1, 5), \quad Adj[5] = (2, 4).$$

Suponha que executamos $BFS(G, 1)$, partindo do vértice 1.

Pede-se: preencha a tabela abaixo com os valores finais de $d[v]$ (distância em arestas até o vértice 1) e $\pi[v]$ (predecessor de v na árvore de busca em largura).

v	1	2	3	4	5
$d[v]$	_____	_____	_____	_____	_____
$\pi[v]$	_____	_____	_____	_____	_____

Exercício Prático

Exercício 2 — Contando vértices alcançáveis a partir de uma origem

Implemente em Java uma classe `AlcancaveisBFS` que, dado um grafo não orientado e um vértice de origem s , conta **quantos vértices** são alcançáveis a partir de s (incluindo o próprio s) usando **busca em largura**.

Requisitos da implementação:

- O grafo deve ser representado por *listas de adjacências*, usando `List<List<Integer>`.
- A classe deve oferecer:
 - um construtor `AlcancaveisBFS(int n)` que cria um grafo com n vértices;
 - um método `adicionarAresta(int u, int v)`;
 - um método `int contarAlcancaveis(int s)` que retorna o número de vértices alcançáveis a partir de s .
- Use uma `Queue<Integer>` (pode ser `LinkedList`) para a fila da BFS.
- O método `main` deve testar a classe sobre um pequeno grafo de exemplo, imprimindo o número de vértices alcançáveis a partir de um vértice escolhido.

Saída esperada: uma linha no formato

Vertices alcancaveis a partir de <s>: <quantidade>